

Prácticas: Información general base del proyecto



Profesor: Galante Diego

Alumnos: Abelleira Diego, Arrúa Francisco

Año: 2025

Contenido

Versión base	3
Estructura y contenido	8
 backend/.....	15
Contenido de cada archivo (simplificado).....	16
 frontend/.....	17
Contenido de cada archivo (simplificado).....	17
¿Nombre?.....	18
Paso a paso	19
Fase 0 — Preparación (crear repositorio y estructura vacía)	19
Fase 1 — Backend: configuración inicial (PRIORIDAD ALTA).....	19
Fase 2 — Base de datos y modelo (PRIORIDAD ALTA).....	20
Fase 3 — Backend: capa modelo / repositorios (PRIORIDAD ALTA).....	20
Fase 4 — Backend: utilidades y middlewares (PRIORIDAD MEDIA).....	21
Fase 5 — Backend: servicios (lógica de negocio) (PRIORIDAD ALTA).....	21
Fase 6 — Backend: controladores y rutas (PRIORIDAD ALTA)	22
Fase 7 — Backend: tests, auditoría y utilidades finales (PRIORIDAD MEDIA)	23
Fase 8 — Frontend: configuración inicial (PRIORIDAD ALTA una vez backend básico listo).....	23
Fase 9 — Frontend: servicios y autenticación (PRIORIDAD ALTA)	24
Fase 10 — Frontend: páginas y componentes básicos (PRIORIDAD ALTA).....	24
Fase 11 — Frontend: validaciones y UX (PRIORIDAD MEDIA).....	25
Fase 12 — Integración, pruebas y ajustes (PRIORIDAD ALTA)	25
Fase 13 — Opcionales (para después).....	25
Orden resumido (lista rápida para ejecutar)	26
Diseño base(Puede cambiar en el futuro).....	26

Versión base

Visión del producto

Una aplicación web para comercios de ropa que centraliza ventas, gastos y liquidaciones de personal para dar una respuesta clara a una pregunta crucial: ¿la tienda está ganando plata y cómo se distribuye esa ganancia entre gastos y sueldos?. Es una herramienta que combina POS ligero + gestión de gastos + cálculo de sueldos/comisiones + reporting visual para tomar decisiones rápidas y confiables.

Objetivo general

Permitir al dueño/gerente conocer en tiempo real la salud financiera de la tienda (ingresos, gastos, sueldos) y automatizar la liquidación de sueldos, el control de presentismo y el seguimiento de metas de ventas.

2 — Alcance y restricciones (claras)

- Usuarios: solo *Contador* (rol admin) y *Administrador*.
- No hay acceso directo para trabajadoras.
- No integración con AFIP ni facturación electrónica.
- Solo funcionalidades mínimas: ventas, gastos, liquidaciones mensuales, informes simples y importación desde Excel/CSV.
- No se implementan workflows complejos (colas, workers, multi-sucursal, POS offline, inventario avanzado).

3 — Objetivos (mínimos)

- Registrar ventas por vendedora (ticket, total, pagos).
- Registrar gastos y asociarlos a un mes (periodo contable).
- Calcular liquidaciones mensuales (sueldo base + comisión simple – presentismo).
- Exportar CSV/PDF de reportes y liquidaciones.
- Importar datos desde Excel/CSV con mapeo sencillo y validación básica.

4 — Stack tecnológico propuesto (simple y probado)

- Frontend: React (SPA). React es la librería estándar para construir interfaces reactivas y SPAs.
- Backend: Node.js + Express (API REST simple). Express es minimalista y adecuado para APIs ligeras.
- Base de datos: PostgreSQL (relacional, fiable para este tipo de datos financieros).
- ORM opcional pero recomendado: Prisma (funciona bien con JavaScript y simplifica consultas/migraciones).

- Herramientas de desarrollo: crear app con Vite.

5 — Arquitectura simplificada

1. Frontend (React SPA) → llamadas a
 2. API REST (Express) → lee/escribe en
 3. Postgres.
- El frontend se sirve como build estático; backend expone endpoints REST.
 - Migraciones / esquema gestionado por Prisma (opcional).
 - No se requieren workers ni colas: las tareas de cálculo se ejecutan on-demand (por petición) o por cron simple si se quiere automatizar.

6 — Modelo de datos

Diseño minimalista, normalizado pero reducido al esencial.

Tablas principales (campos clave)

1. usuarios
 - id (PK), nombre, email (único), password_hash, rol, creado_en
2. vendedoras
 - id (PK), nombre, codigo (opcional), sueldo_base (numeric), porcentaje_comision (numeric), creado_en
3. ventas
 - id (PK), fecha (date), hora (time opcional), vendedora_id (FK), total (numeric), estado (finalizada/devolucion), ticket_num (nullable, único opcional), saldo_pendiente (numeric), batch_id (nullable)
4. venta_items
 - id (PK), venta_id (FK), descripcion, cantidad (int), precio_unitario (numeric), descuento (numeric)
5. pagos_venta
 - id (PK), venta_id (FK), metodo_pago (efectivo/tarjeta/transferencia), monto (numeric), referencia (string nullable)
6. gastos
 - id (PK), fecha, monto (numeric), categoria (string), descripcion, comprobante_url (nullable), periodo (YYYY-MM), creado_por (FK usuarios)
7. asistencias
 - id (PK), vendedora_id (FK), fecha, presente (boolean), motivo (nullable)
8. liquidaciones

- id (PK), vendedora_id (FK), periodo (YYYY-MM), sueldo_base, comisiones, presentismo_descuento, bonos (numeric), total_pagar, estado (generada/pagada), generado_por (FK usuarios), creado_en
9. import_batches
- id (PK), usuario_id (FK), fecha, archivo_nombre, filas_totales, filas_importadas, filas_fallidas, estado
10. auditoria
- id (PK), entidad (string), entidad_id, accion (create/update/delete), usuario_id, timestamp, datos_antes JSON, datos_despues JSON

Relaciones

- vendedoras 1—N ventas
- ventas 1—N venta_items y 1—N pagos_venta
- vendedoras 1—N asistencias y 1—N liquidaciones
- import_batches referencia importaciones masivas (Excel)

Nota: el esquema suprime tablas de desglose de liquidación y ajustes complejos (las liquidaciones contienen valores agregados y se guarda referencia a ventas si hace falta en campos adicionales).

7 — Endpoints REST mínimos (contratos conceptuales)

- POST /api/auth/login — login contadora.
- GET /api/vendedoras — listar.
- POST /api/ventas — crear venta (items + pagos).
- GET /api/ventas?start&end&vendedora — listar/filtrar (paginado).
- POST /api/ventas/:id/devolucion — registrar devolución simple.
- POST /api/gastos — crear gasto.
- GET /api/gastos?periodo — listar gastos.
- POST /api/import — subir Excel/CSV (simple mapeo por cabeceras).
- POST /api/liquidaciones/generar — generar liquidaciones para periodo (preview).
- POST /api/liquidaciones/:id/confirmar — confirmar y persistir liquidación.
- GET /api/reportes/ganancia?start&end — resumen simple: ventas, gastos, sueldos, ganancia.
- GET /api/export/csv?tipo=ventas — export.

8 — Flujos simplificados (qué hace la app, paso a paso)

A) Registrar una venta

1. La contadora abre “Nueva venta”, elige la vendedora.
2. Ingresa items (opcional) y el total; registra método de pago (puede ser parcial).
3. Al confirmar: backend valida (fecha, $\text{total} \geq \text{suma items}$, vendedor existe) y guarda ventas, venta_items, pagos_venta.
4. Se asigna ticket_num (si se desea) y se guarda batch_id si viene por import.

B) Registrar un gasto

1. Form minimal: fecha, categoria, monto, descripción, adjuntar comprobante (opcional).
2. Se guarda y se puede filtrar por periodo.

C) Generar liquidaciones (mensual)

1. La contadora solicita “Generar liquidaciones” para YYYY-MM.
2. Backend obtiene ventas de cada vendedora en ese periodo, calcula comisiones: $\text{SUM}(\text{ventas}) * \text{porcentaje_comision} / 100$.
3. Revisa asistencias: si regla de presentismo indica deducción, aplica presentismo_descuento.
4. $\text{total_pagar} = \text{sueldo_base} + \text{comisiones} + \text{bonos} - \text{presentismo_descuento}$.
5. Devuelve preview por vendedora; al confirmar, persiste la liquidacion con estado generada. Si se paga, cambiar a pagada.

D) Importar desde Excel/CSV (modo simple)

1. La contadora sube archivo; sistema intenta mapear por cabeceras (nombre exacto) y muestra una vista previa de N filas.
2. Validaciones básicas: tipos (fecha, número), vendedor coincidente o crear vendedora nueva si se decide.
3. Confirmación → inserta registros en una transacción o por batch (si es muy grande). Registra import_batches.

9 — Validaciones y reglas de negocio (mínimas)

- $\text{venta.total} \geq \text{suma de venta_items}$.
- $\text{pagos_venta sumados} \leq \text{venta.total}$.
- ticket_num único si se usa (evitar duplicados).
- presentismo_descuento por regla simple (ej.: si faltas > 1 → descuento X% del sueldo_base).
- Devoluciones: crear registro que reduce la comisión proporcionalmente.

10 — Seguridad y operación básica

- Autenticación: sesiones seguras con cookies + hash de password (bcrypt).

- Autorización: endpoints protegidos por rol.
- Backups: backup diario Postgres (snapshot).
- HTTPS obligatorio en producción.
- Auditoría mínima: registrar creación/edición/eliminación en auditoria.
- No implementar complejas políticas de retención ahora (puede decidirse luego).

11 — Despliegue mínimo viable

- DB: managed Postgres (Supabase / DigitalOcean).
- Backend: despliegue simple en un servicio (e.g. Render / Railway) o VPS; sirve API y opcionalmente build estático.
- Frontend: build estático servido por Nginx o por el mismo backend.
- Backups y logs básicos; no se requieren colas ni Redis.

12 — MVP: criterios de aceptación (lo mínimo que debe funcionar)

1. Login y control de acceso.
2. Registrar ventas y listar/filtrar por vendedora/fecha.
3. Registrar gastos y listar por periodo.
4. Generar preview de liquidaciones mensuales y confirmar liquidación.
5. Importar Excel/CSV con mapping simple y ver resumen de import.
6. Exportar CSV de ventas/gastos/liquidaciones.
7. Auditoría básica de acciones.

13 — Qué se removió de la versión completa (para simplificar)

- No hay control de multi-sucursal ni POS offline.
- No hay workers/colas (Bull/Redis) ni jobs complejos; cálculos se hacen a demanda.
- No vistas/materialized views ni optimizaciones OLAP: consultas simples con índices.
- No módulo avanzado de metas/buenificaciones complejas (solo bonificación simple si se quiere).
- No motor de reglas para mapeo avanzado de Excel — sólo mapeo por cabeceras + vista previa.
- Menos tablas de desagregación (liquidaciones menos detalladas; fewer audit fields).

14 — Ventajas de esta versión simple

- Menos tiempo de desarrollo (menos componentes y menos infra).

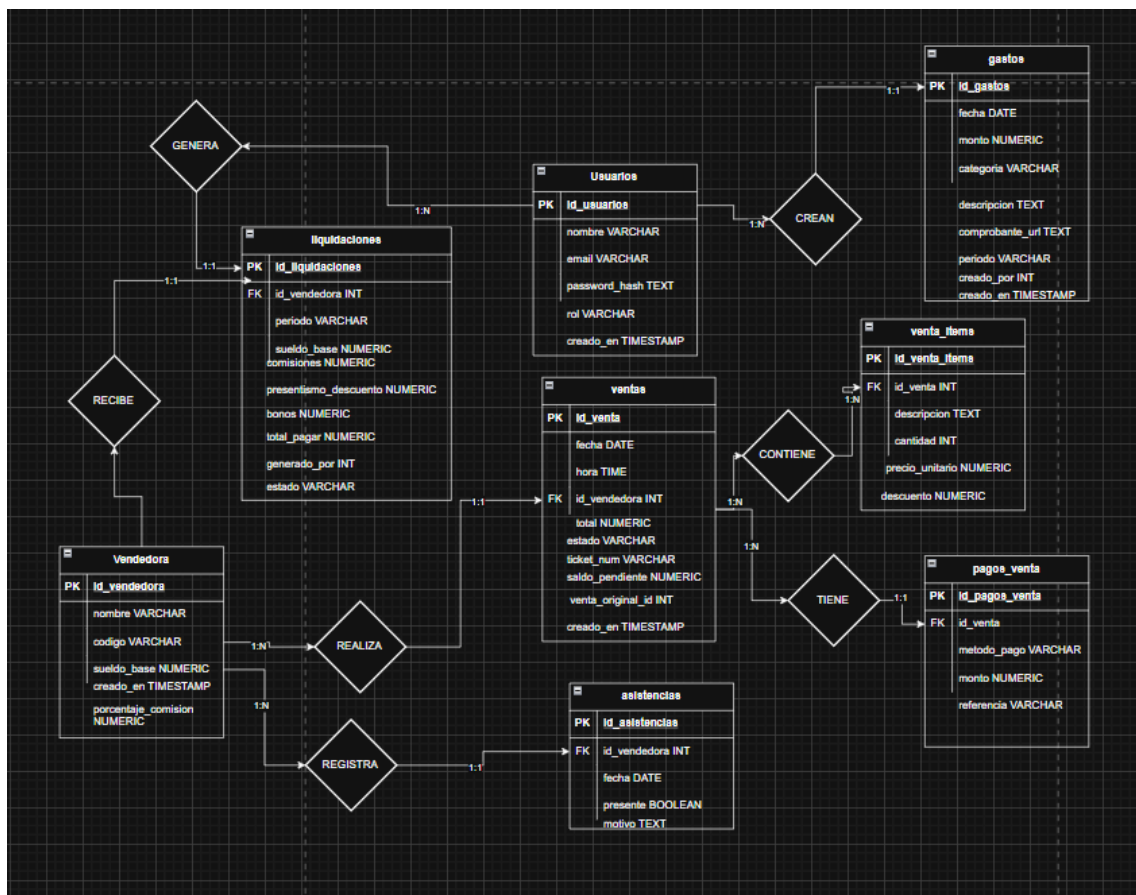
- Más fácil de mantener para un equipo chico o un dev freelance.
- Cumple las necesidades principales de la contadora sin complejidad extra.

15 — Sigüientes pasos recomendados

1. Validar con la contadora las reglas de presentismo y comisiones exactas (por escrito).
2. Definir plantilla simple de import (cabeceras esperadas) para agilizar las importaciones.
3. Implementar el esquema de DB compacto (tablas arriba) y crear endpoints mínimos.
4. Hacer pruebas con 3-5 archivos reales de Excel/CSV y un dataset de ejemplo.
5. Iterar: si se necesita más funcionalidad, añadir modularmente (p.ej. worker para jobs, desglose de liquidaciones).

Estructura y contenido

Base de datos



Entidades: **usuarios**, **vendedoras**, **ventas**, **venta_items**, **pagos_venta**, **gastos**, **asistencias**, **liquidaciones**.

Relaciones claves: una **vendedora** tiene muchas **ventas**, cada **venta** tiene muchos **venta_items** y **pagos_venta**; las **liquidaciones** se calculan por vendedora y periodo usando ventas + asistencias; **usuarios** crean gastos y generan liquidaciones.

1) usuarios

Propósito: representar cuentas que operan la app (admin / contadora).

Campos principales

- id (SERIAL PK) — identificador.
- nombre (VARCHAR) — nombre visible.
- email (VARCHAR, UNIQUE) — login.
- password_hash (TEXT) — bcrypt (no guardar texto plano).
- rol (VARCHAR, CHECK IN ('admin','contador')) — autorización.
- creado_en (TIMESTAMP) — fecha/hora creación.

Relaciones

- usuarios 1 — N gastos (gastos.creado_por → usuarios.id).
- usuarios 1 — N liquidaciones (liquidaciones.generado_por → usuarios.id) — registra quién generó la liquidación.

Reglas / notas

- Indexar email (único) para login.
 - No usar usuario como FK restrictivo si querés mantener históricos: emplear ON DELETE SET NULL para campos creados_por/generado_por (así no se borra info crítica si se elimina un usuario).
 - Registrar intentos de login y bloqueos en la capa aplicación (no tenemos auditoría en BD).
-

2) vendedoras

Propósito: las empleadas que generan ventas y reciben liquidaciones.

Campos principales

- id (PK)
- nombre (VARCHAR)
- codigo (VARCHAR UNIQUE) — identificador opcional para import/etiquetas.
- sueldo_base (NUMERIC(12,2)) — salario fijo a considerar en liquidación.
- porcentaje_comision (NUMERIC(5,2)) — % aplicado sobre ventas del periodo.
- creado_en (TIMESTAMP)

Relaciones

- vendedoras 1 — N ventas (ventas.vendedora_id).

- vendedoras 1 — N asistencias (asistencias.vendedora_id).
- vendedoras 1 — N liquidaciones (liquidaciones.vendedora_id).

Reglas / notas

- porcentaje_comision entre 0 y 100 (CHECK).
 - ON DELETE RESTRICT para vendedoras en la FK de ventas, para evitar perder historial; si de todas formas querés permitir borrado, usar soft-delete en vez de borrar registros históricos.
 - Index recomendado: vendedoras(codigo).
-

3) ventas

Propósito: registro de cada transacción comercial (ticket).

Campos principales

- id (PK)
- fecha (DATE) — día de la venta (usado para periodos).
- hora (TIME) — opcional.
- vendedora_id (FK → vendedoras.id) — la responsable de la venta.
- total (NUMERIC(12,2)) — monto total declarado de la venta (≥ 0).
- estado (VARCHAR) — finalizada o devolucion.
- ticket_num (VARCHAR UNIQUE) — número de comprobante si lo hay.
- saldo_pendiente (NUMERIC) — si hay pagos parciales.
- venta_original_id (FK → ventas.id, NULLABLE) — referencia si la venta es una devolución de otra.
- creado_en (TIMESTAMP)

Relaciones

- ventas N — 1 vendedoras
- ventas 1 — N venta_items (venta_items.venta_id).
- ventas 1 — N pagos_venta (pagos_venta.venta_id).
- ventas 0..1 — 1 ventas (auto-FK para devoluciones: venta_original_id).

Reglas / notas

- **Invariante lógico importante:** venta.total ≥ 0 .
Para devoluciones: mantén estado = 'devolucion' y referencia venta_original_id. NO es necesario que total sea negativo; en los cálculos de comisiones y totales de periodo restarás ventas con estado = 'devolucion'. Esto evita violar CHECK (total ≥ 0) y mantiene trazabilidad.

- **Consistency rules (implementar en backend):**
 - total debe ser igual o mayor a $\text{SUM}(\text{venta_items.cantidad} * \text{precio_unitario} - \text{descuento})$ — validar al crear/editar.
 - $\text{SUM}(\text{pagos_venta.monto}) \leq \text{total}$ — validar al crear/editar pagos.
- ON DELETE RESTRICT en vendedora_id (no permitir borrar si hay ventas).
- Índices recomendados: ventas(fecha), ventas(vendedora_id, fecha), ticket_num (unique).

Operaciones típicas

- Crear venta (insert venta → insert items → insert pagos) dentro de una transacción DB.
- Registrar devolución: crear nueva fila con estado='devolucion' + venta_original_id apuntando a la venta devuelta.

4) venta_items

Propósito: detalle de cada producto/servicio incluido en una venta.

Campos principales

- id (PK)
- venta_id (FK → ventas.id) — ON DELETE CASCADE
- descripcion (TEXT)
- cantidad (INT > 0)
- precio_unitario (NUMERIC)
- descuento (NUMERIC >= 0)
- creado_en (TIMESTAMP)

Relaciones

- venta_items N — 1 ventas

Reglas / notas

- ON DELETE CASCADE para venta_items facilita borrar todo el detalle cuando se elimina una venta (en desarrollo local). En producción preferir soft-delete.
- Validar que $\text{cantidad} * \text{precio_unitario} - \text{descuento} \geq 0$.
- No hay relación con inventario en este modelo (no trackeamos stock).

5) pagos_venta

Propósito: registrar los pagos aplicados a cada venta (puede haber varios métodos por venta).

Campos principales

- id (PK)
- venta_id (FK → ventas.id) — ON DELETE CASCADE
- metodo_pago (VARCHAR CHECK IN ('efectivo','tarjeta','transferencia')) — ampliar según necesites.
- monto (NUMERIC >= 0)
- referencia (VARCHAR) — id de transacción, etc.
- creado_en (TIMESTAMP)

Relaciones

- pagos_venta N — 1 ventas

Reglas / notas

- SUM(pagos_venta.monto) debería ser $\leq$ venta.total.
 - Para conciliación de caja: agregar campo conciliado o registro_caja_id si más tarde implementás conciliación con reporte contable.
-

6) gastos

Propósito: almacenar gastos operativos (restan de la ganancia del periodo).

Campos principales

- id (PK)
- fecha (DATE)
- monto (NUMERIC >= 0)
- categoria (VARCHAR) — ej. alquiler, servicios, insumos.
- descripcion (TEXT)
- comprobante_url (TEXT) — link al PDF/foto (opcional).
- periodo (VARCHAR(7), formato YYYY-MM) — para agregados por mes.
- creado_por (FK → usuarios.id) — quién registró.
- creado_en (TIMESTAMP)

Relaciones

- gastos N — 1 usuarios (creado_por)

Reglas / notas

- Mantener periodo consistente con fecha (por ejemplo, periodo = to_char(fecha, 'YYYY-MM')) — validalo en backend o con trigger.

- Índice recomendado: gastos(periodo) para reportes por mes.
-

7) asistencias

Propósito: control diario de presentismo por vendedora (presente/falta).

Campos principales

- id (PK)
- vendedora_id (FK → vendedoras.id) — ON DELETE CASCADE
- fecha (DATE)
- presente (BOOLEAN) — true/false
- motivo (TEXT)
- UNIQUE(vendedora_id, fecha) para evitar duplicados.

Relaciones

- asistencias N — 1 vendedoras

Reglas / notas

- Se usa para calcular descuentos por presentismo en liquidaciones.
 - Cálculo típico: contar presente = false en el periodo; aplicar regla de descuento definida por la contadora (p. ej. si faltas > 1 entonces X% del sueldo base).
 - Index recomendados: (vendedora_id, fecha).
-

8) liquidaciones

Propósito: resultado del cálculo mensual del sueldo de cada vendedora.

Campos principales

- id (PK)
- vendedora_id (FK → vendedoras.id) — ON DELETE RESTRICT
- periodo (VARCHAR(7), YYYY-MM) — mes de la liquidación
- sueldo_base (NUMERIC)
- comisiones (NUMERIC) — calculadas: SUM(ventas relevantes) * porcentaje
- presentismo_descuento (NUMERIC)
- bonos (NUMERIC) — ajustes manuales/extra
- total_pagar (NUMERIC) — sueldo_base + comisiones + bonos – presentismo_descuento
- estado (VARCHAR CHECK IN ('generada','confirmada','pagada'))

- generado_por (FK → usuarios.id) — quién generó la liquidación (opcional)
- UNIQUE (vendedora_id, periodo) — una liquidación por mes por vendedora

Relaciones

- liquidaciones N — 1 vendedoras
- liquidaciones N — 1 usuarios (generado_por)

Reglas / notas

- **Cálculo (algoritmo general):**
 1. Obtener ventas del periodo para la vendedora: WHERE fecha BETWEEN periodo_start AND periodo_end.
Incluir ventas normales; restar estado='devolucion'.
 2. $\text{comisiones} = \text{SUM}(\text{ventas_positivas}) * (\text{vendedoras.porcentaje_comision} / 100) - \text{SUM}(\text{devoluciones_aplicables}) * (\text{porcentaje}/100)$
(simplificación: restar ventas de devoluciones).
 3. presentismo_descuento = aplicar regla en función de asistencias (ej. si faltas > N => descuento X% del sueldo_base, o descuento fijo por falta).
 4. $\text{total_pagar} = \text{sueldo_base} + \text{comisiones} + \text{bonos} - \text{presentismo_descuento}.$
- Guardar sueldo_base y porcentaje_comision en la liquidación (valores “snapshotados”) para auditoría histórica (si luego cambian los datos en vendedoras, la liquidación conserva los números).
- Realizar generación en modo **preview** antes de persistir (backend devuelve desglose por vendedor y permite confirmación).

Reglas de integridad y validaciones que debe imponer el backend (imprescindible)

1. **Transacciones:** al crear una venta: insertar venta → insertar items → insertar pagos dentro de una única transacción DB para evitar estados parciales.
2. **Comprobaciones numéricas:** validar $\text{venta.total} \geq \text{SUM}(\text{items})$ y $\text{SUM}(\text{pagos}) \leq \text{venta.total}$ antes de commit. Rechazar o corregir errores.
3. **Devoluciones:** crear nueva venta con estado='devolucion' y venta_original_id apuntando a la venta original; al calcular comisiones restar dichas devoluciones.
4. **Liquidaciones:** calcular siempre desde datos de ventas y asistencias; guardar resultados calculados en liquidaciones para trazabilidad.
5. **UNIQUE en liquidaciones:** impedir generación duplicada para la misma vendedora_id y periodo.
6. **Formato de periodo en gastos/liquidaciones:** validar YYYY-MM.

7. **Manejo de borrados:** preferir soft-delete si necesitas conservar históricos; si usas borrados reales, usar ON DELETE apropiado (RESTRICT vs CASCADE vs SET NULL) según lo definido.

backend/

Carpeta donde vive la API en Node.js + Express.

```
backend/
├── src/
│   ├── config/
│   │   └── db.js
│   ├── models/
│   │   ├── Empleado.js
│   │   ├── Venta.js
│   │   └── Usuario.js
│   ├── controllers/
│   │   ├── empleadoController.js
│   │   ├── ventaController.js
│   │   └── usuarioController.js
│   ├── services/
│   │   ├── empleadoService.js
│   │   ├── ventaService.js
│   │   └── usuarioService.js
│   ├── repositories/
│   │   ├── empleadoRepository.js
│   │   ├── ventaRepository.js
│   │   └── usuarioRepository.js
│   ├── routes/
│   │   ├── empleadoRoutes.js
│   │   ├── ventaRoutes.js
│   │   └── usuarioRoutes.js
│   ├── middlewares/
│   │   └── authMiddleware.js
│   ├── utils/
│   │   └── excelParser.js
│   ├── uploads/
│   │   └── (vacío, se llena con los excels subidos)
│   └── app.js
├── package.json
└── .env
```

Contenido de cada archivo (simplificado)

config/db.js

- Configura la conexión a la base de datos (por ejemplo, MySQL o PostgreSQL).
- Usa variables de entorno de `.env`.

models/

- `Empleado.js` → Define la tabla/colección empleados (nombre, apellido, puesto, presentismo, etc.).
- `Venta.js` → Define la tabla/colección ventas (fecha, monto, empleadoId, etc.).
- `Usuario.js` → Define la tabla usuarios para el login (usuario, contraseña encriptada, rol).

controllers/

- Reciben las peticiones HTTP, validan parámetros y llaman a services.
- Devuelven la respuesta al cliente.
- Ejemplo: `ventaController.js` maneja POST para subir excel, GET para listar ventas.

services/

- Contienen la lógica de negocio.
- Ejemplo: `ventaService.js` calcula totales, verifica presentismo.
- Lllaman a repositories para acceder a datos.

repositories/

- Manejan la interacción directa con la base de datos.
- Ejemplo: `ventaRepository.js` tiene funciones `crearVenta()`, `obtenerVentasPorEmpleado()`.

routes/

- Define las rutas de la API y asigna qué controlador se ejecuta.
- Ejemplo: `/ventas/upload` usa `ventaController.subirExcel`.

middlewares/

- Funciones que se ejecutan antes de llegar al controlador.
- `authMiddleware.js` valida si el usuario está logueado y autorizado.

utils/

- Funciones auxiliares.

- excelParser.js → Detecta formato del Excel subido, mapea columnas, genera un formato estándar.

uploads/

- Carpeta temporal donde se guardan los archivos Excel antes de procesarlos.

app.js

- Configuración de Express.
- Conexión a DB.
- Uso de middlewares globales.
- Registro de rutas.

frontend/

Aplicación en React.js.

```
frontend/
├── public/
│   └── index.html
├── src/
│   ├── components/
│   │   ├── Navbar.jsx
│   │   ├── VentaTable.jsx
│   │   └── UploadExcelForm.jsx
│   ├── pages/
│   │   ├── LoginPage.jsx
│   │   ├── DashboardPage.jsx
│   │   └── VentasPage.jsx
│   ├── services/
│   │   └── api.js
│   ├── utils/
│   │   └── formatData.js
│   ├── App.js
│   └── index.js
├── package.json
└── .env
```

Contenido de cada archivo (simplificado)

components/

- Navbar.jsx → Menú superior de navegación.

- VentaTable.jsx → Muestra ventas en una tabla.
- UploadExcelForm.jsx → Formulario para subir Excel.

pages/

- LoginPage.jsx → Pantalla de login para el contador.
- DashboardPage.jsx → Resumen de ventas, presentismo, totales.
- VentasPage.jsx → Listado de ventas filtrables por empleado y fecha.

services/api.js

- Centraliza llamadas a la API con fetch o axios.

utils/formatData.js

- Funciones para dar formato a datos (ej: fechas, montos).

App.js

- Configura las rutas de la app con React Router.

index.js

- Punto de entrada del frontend.

¿Nombre?

Rendix.

Tiene un sonido moderno y profesional, lo que lo hace adecuado para una aplicación enfocada en la optimización de la rentabilidad de una tienda. Su nombre evoca "rendimiento" y "eficiencia", lo que conecta directamente con la propuesta de valor del producto: mejorar la rentabilidad de los comercios a través de un control claro de ventas, gastos, sueldos y comisiones.

Potenciales significados de Rendix:

1. Rendimiento + X: La "X" puede representar lo desconocido, la variable a resolver. Alude a que la herramienta resuelve el enigma de saber si un negocio está siendo rentable y cómo distribuir esa rentabilidad (ventas, sueldos, gastos). El "X" también puede aludir a algo avanzado o de tecnología, lo cual hace que el nombre suene más innovador.
2. Rendimiento Exitoso: En inglés, "rend" (de render) hace referencia a producir o entregar un resultado. "Rendix" podría ser interpretado como una herramienta que entrega rendimiento al comercio o negocio, lo que da una sensación de eficiencia y resultados medibles.
3. Optimización de Rentabilidad: Rendix claramente comunica la optimización de las ganancias en los comercios. "Rendimiento" es un término clave en negocios, y este nombre lo condensa de manera simple.

Potenciales Slogans para Rendix:

- "Rendix: Controla tu rentabilidad al instante."
- "Optimiza cada venta, cada gasto, cada sueldo con Rendix."
- "Donde tus ventas se encuentran con la rentabilidad."
- "Simplifica tu tienda, maximiza tus ganancias."
- "Rendix: Tu socio para un comercio más rentable."

Características Visuales del Branding:

- Logo: Podría tener una tipografía moderna, limpia y minimalista, jugando con la "X" al final como un símbolo de precisión o punto de interacción.
- Colores: Colores como el verde (crecimiento, éxito), azul (confianza, profesionalismo) o incluso un gris oscuro (tecnología, sofisticación) podrían ser apropiados.
- Iconografía: Un gráfico de barras o un círculo (representando fluidez y optimización) podría ser parte del logo o los íconos dentro de la app.

Paso a paso

Fase 0 — Preparación (crear repositorio y estructura vacía)

Crear estructura de carpetas vacía

```
/proyecto-pos-contador
├── backend/
└── frontend/
```

Fase 1 — Backend: configuración inicial (PRIORIDAD ALTA)

Objetivo: tener servidor Express arrancando y con DB accesible.

1. backend/package.json

- Contenido: metadatos del proyecto y scripts (dev, start, test, lint).
- ¿Por qué primero?: Se define dependencias y scripts que vamos a usar.

2. backend/.env (local, no en Git)

- Variables: DB connection string, PORT, JWT_SECRET, UPLOAD_DIR, etc.
- ¿Por qué?: necesarios para configurar DB/server desde el inicio.

3. backend/src/app.js (o server.js)

- Contenido: inicialización de Express, middlewares básicos (JSON parser, CORS), import de rutas, arranque del servidor en puerto.
- Test: levantar servidor y comprobar response básico (200 OK).

4. backend/src/config/db.js

- Contenido: función/objeto para conectar a la BD (pool o cliente). Usa variables de .env.
- ¿Por qué ahora?: las capas siguientes dependerán de la conexión DB.

5. backend/README-DEV.md (opcional pero recomendable)

- Contenido: pasos para instalar dependencias, variables .env necesarias, cómo arrancar el servidor.

Fase 2 — Base de datos y modelo (PRIORIDAD ALTA)

Objetivo: tener tablas/migraciones listas para persistencia.

6. backend/prisma/schema.prisma (opcional) o backend/migrations/ (SQL)**

- Contenido: esquema DB (tablas minimalistas: usuarios, vendedoras, ventas, venta_items, pagos_venta, gastos, asistencias, liquidaciones, import_batches, auditoria).
- ¿Por qué ahora?: definir estructura antes de implementar repositorios.
- Si no usamos Prisma: colocamos scripts SQL en backend/db/migrations/.

7. Ejecutar migraciones / crear tablas

- Acción (no archivo): aplicar las migraciones a la BD y verificar tablas creadas.

Fase 3 — Backend: capa modelo / repositorios (PRIORIDAD ALTA)

Objetivo: encapsular acceso a la BD.

8. backend/src/models/ (*crear archivos por entidad*)

- src/models/usuarioModel.js — definiciones de acceso (si usamos ORM, modelos; si usamos queries, helpers).
- src/models/vendedoraModel.js
- src/models/ventaModel.js
- src/models/ventaItemModel.js
- src/models/pagoVentaModel.js
- src/models/gastoModel.js
- src/models/asistenciaModel.js

- `src/models/liquidacionModel.js`
- `src/models/importBatchModel.js`
- `src/models/auditoriaModel.js`
- Contenido: funciones CRUD básicas (create, findById, findAll with filters).
- ¿Por qué ahora?: servicios y controladores los usarán.

9. `backend/src/repositories/` (si separas repositorios de modelos)

- Si prefieres separar: `ventaRepository.js`, `gastoRepository.js`, etc. Aquí van las queries concretas (joins/aggregates).
- Importante: definir funciones que recibirán parámetros claros (ej. `getVentasByPeriod({start, end, vendedorId})`).

Fase 4 — Backend: utilidades y middlewares (PRIORIDAD MEDIA)

Objetivo: tener herramientas compartidas.

10. `backend/src/middlewares/errorHandler.js`

- Contenido: manejo centralizado de errores y formato de respuesta.
- ¿Por qué?: mejora la estabilidad desde el inicio.

11. `backend/src/middlewares/authMiddleware.js`

- Contenido: verificación de sesión/JWT, extraer usuario en `req.user`.
- ¿Por qué antes de rutas protegidas**: vas a proteger endpoints.

12. `backend/src/utils/excelParser.js`

- Contenido: lectura de Excel/CSV, normalización básica (trim, parseo de fechas, numeros).
- ¿Por qué ahora?: importer endpoint lo usará pronto.

13. `backend/src/utils/fileHelpers.js`

- Contenido: manejo de uploads temporales, limpieza, generación de `batch_id` y cálculo de hash del archivo.

Fase 5 — Backend: servicios (lógica de negocio) (PRIORIDAD ALTA)

Objetivo: centralizar reglas (sueldo, comisiones, presentismo).

14. `backend/src/services/ventaService.js`

- Contenido: lógica para crear venta (validaciones), calcular saldo pendiente, marcar devoluciones y crear auditoría.
- Usará modelos/repositorios.

15. backend/src/services/gastoService.js

- Contenido: crear gasto, validar periodo contable, guardar comprobante URL.

16. backend/src/services/liquidacionService.js

- Contenido: función que calcula preview de liquidaciones por periodo para cada vendedora (sueldo_base + comisiones – presentismo).
- Debe devolver desglose para UI.

17. backend/src/services/importService.js

- Contenido: lógica que orquesta Excel → parse → validate → map → persist por batch. Manejo de duplicados, creación de vendedoras provisionales, creación de import_batches y grabado de auditoría.

Fase 6 — Backend: controladores y rutas (PRIORIDAD ALTA)

Objetivo: exponer la API REST mínima.

18. backend/src/controllers/usuarioController.js

- Contenido: login, logout, crear usuario (solo admin).

19. backend/src/controllers/vendedoraController.js

- Contenido: listar, crear/edit vendedora.

20. backend/src/controllers/ventaController.js

- Contenido: crear venta (desde formulario), endpoint para subir Excel (/ventas/import), listar/filtrar ventas, endpoint de devolución.

21. backend/src/controllers/gastoController.js

- Contenido: crear/listar gastos.

22. backend/src/controllers/liquidacionController.js

- Contenido: generar preview (/liquidaciones/generar), confirmar (/liquidaciones/:id/confirmar), listar.

23. backend/src/routes/index.js (o carpeta routes/)

- Contenido: montar routers: /api/auth, /api/vendedoras, /api/ventas, /api/gastos, /api/liquidaciones. Aplicar authMiddleware donde corresponda.

24. backend/src/app.js (actualizar)

- Ya creado en Fase1; ahora registrar rutas y middlewares finales.
-

Fase 7 — Backend: tests, auditoría y utilidades finales (PRIORIDAD MEDIA)

Objetivo: asegurar calidad básica.

25. backend/tests/

- Tests unitarios para liquidacionService y ventaService (casos de comisión, presentismo, devoluciones).
- Tests de integración para endpoints principales (crear venta, listar).

26. backend/src/auditoria/ or src/services/auditService.js

- Contenido: funciones para registrar acción (entidad, id, usuario, antes/despues, motivo). Usado por controladores y servicios.

27. backend/scripts/seed.js (opcional)

- Contenido: script para poblar DB con ejemplo (vendedoras, algunas ventas) para pruebas.
-

Fase 8 — Frontend: configuración inicial (PRIORIDAD ALTA una vez backend básico listo)

Objetivo: tener SPA que consuma los endpoints básicos.

28. frontend/package.json

- Definir scripts (start, build, test).

29. frontend/.env

- REACT_APP_API_URL, otros settings.

30. frontend/src/index.js

- Punto de entrada, render de App.

31. frontend/src/App.js

- Router básico con rutas: /login, /dashboard, /ventas, /gastos, /liquidaciones.
-

Fase 9 — Frontend: servicios y autenticación (PRIORIDAD ALTA)

Objetivo: llamadas centralizadas y gestión de sesión.

32. frontend/src/services/api.js

- Contenido: wrapper de fetch/axios con baseURL, manejo de tokens, interceptors para 401.

33. frontend/src/context/AuthContext.js (o hook useAuth)

- Contenido: guardar token y datos del usuario, funciones login/logout.
-

Fase 10 — Frontend: páginas y componentes básicos (PRIORIDAD ALTA)

Objetivo: interfaz mínima usable por la contadora.

34. frontend/src/pages/LoginPage.jsx

- Formulario de login y redirect a dashboard.

35. frontend/src/pages/DashboardPage.jsx

- Muestra KPIs (totales del periodo) llamando a /api/reportes/ganancia.

36. frontend/src/pages/VentasPage.jsx

- Tabla de ventas (component VentaTable), botón "Subir Excel".

37. frontend/src/pages/GastosPage.jsx

- Form para nuevo gasto y tabla de gastos.

38. frontend/src/pages/LiquidacionesPage.jsx

- Botón para generar preview y listado de liquidaciones.

- 39. frontend/src/components/Navbar.jsx
- 40. frontend/src/components/VentaTable.jsx
- 41. frontend/src/components/UploadExcelForm.jsx

- Contenido: componentes UI, formularios y llamadas a api.js.
-

Fase 11 — Frontend: validaciones y UX (PRIORIDAD MEDIA)

Objetivo: buena experiencia básica.

- 42. frontend/src/utils/formatData.js

- Funciones para formatear moneda, fechas.

- 43. frontend/src/hooks/useFetch.js (opcional)

- Hook reutilizable para llamadas y estados loading/error.
-

Fase 12 — Integración, pruebas y ajustes (PRIORIDAD ALTA)

Objetivo: conectar y validar flujo completo.

- 44. Integración manual

- Pasos: arrancar backend, crear usuario admin, arrancar frontend, login, crear vendedora manual o importar Excel de prueba, crear venta, generar liquidación preview y confirmar.

- 45. frontend/README.md & backend/README.md

- Documentación mínima de cómo ejecutar, variables .env y endpoints principales.
-

Fase 13 — Opcionales (para después)

- 46. backend/src/jobs/cron.js (si queremos automatizar generación de liquidaciones).

- 47. backend/src/services/pdfService.js (generar PDF de liquidaciones con Puppeteer).

- 48. backend/docs/openapi.yaml (documentar la API).

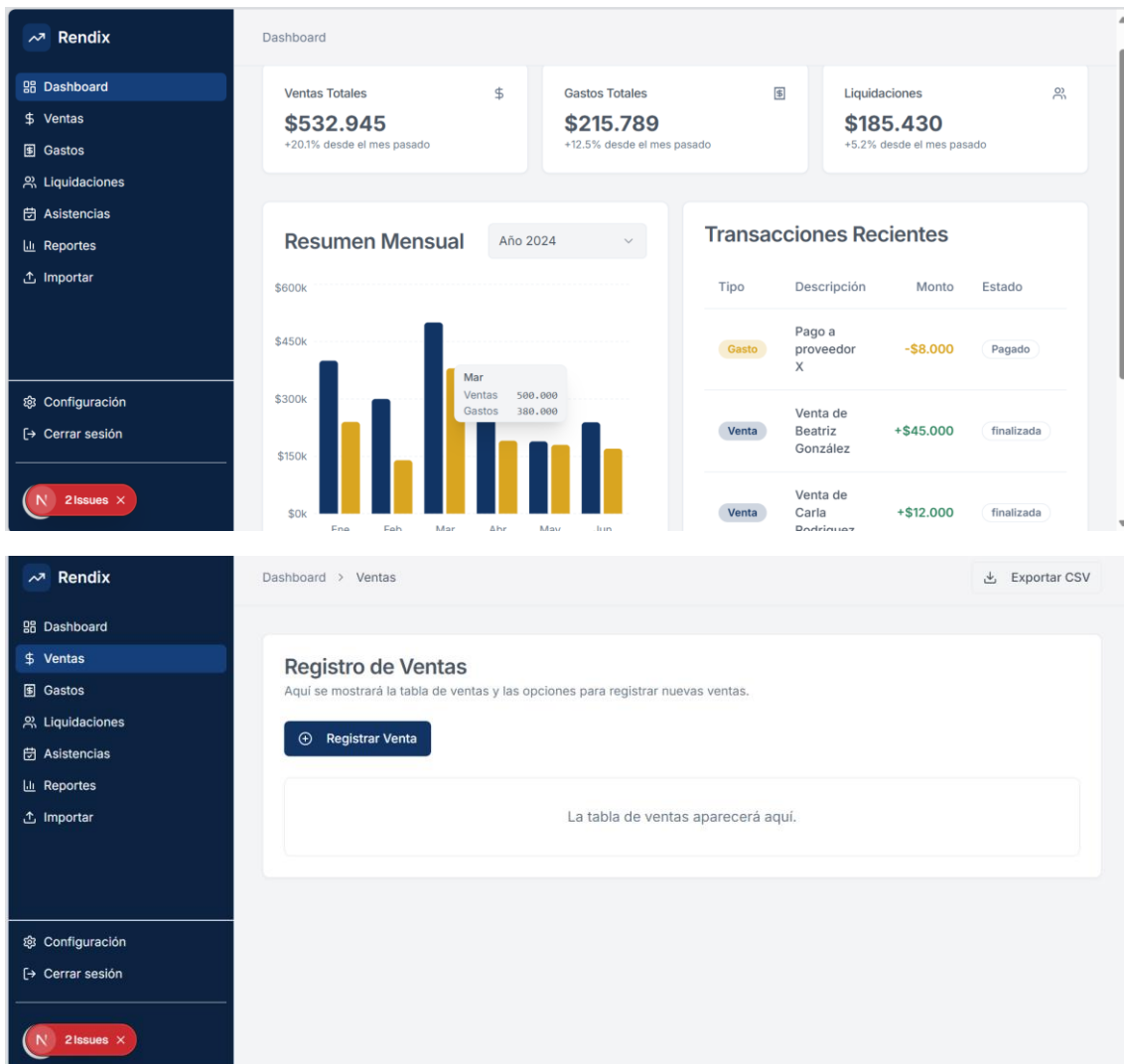
- 49. frontend/src/styles/ (CSS/Tailwind config).

- 50. docker-compose.yml (contenedores para local: db + app).
-

Orden resumido (lista rápida para ejecutar)

1. backend/package.json, .env
2. backend/src/app.js, backend/src/config/db.js
3. Crear migraciones / tablas (Prisma schema o SQL)
4. backend/src/models/* y/o repositories/*
5. backend/src/middlewares/* y src/utills/* (excelParser)
6. backend/src/services/* (ventaService, liquidacionService, importService)
7. backend/src/controllers/* y backend/src/routes/*
8. Probar endpoints con Postman (crear vendedora, crear venta)
9. frontend/package.json, frontend/src/index.js, frontend/src/App.js
10. frontend/src/services/api.js y AuthContext
11. frontend páginas y componentes (Login, Dashboard, Ventas, UploadForm)
12. Integración end-to-end y ajustes
13. Tests y documentación (README, seeds)
14. Opcionales: PDF, cron jobs, Docker

Diseño base(Puede cambiar en el futuro)



 **Rendix**

Dashboard

Ventas

Gastos

Liquidaciones

Asistencias

Reportes

Importar

Configuración

Cerrar sesión

N

2 Issues

X

Dashboard > Gastos

Exportar CSV

Registro de Gastos

Aquí se mostrará la tabla de gastos y las opciones para registrar nuevos gastos.

+

 Registrar Gasto

La tabla de gastos aparecerá aquí.

 **Rendix**

Dashboard

Ventas

Gastos

Liquidaciones

Asistencias

Reportes

Importar

Configuración

Cerrar sesión

N

2 Issues

X

Dashboard > Liquidaciones

Exportar CSV

Liquidación de Sueldos

Aquí se mostrará la lista de vendedoras para calcular sus liquidaciones mensuales.

📄

 Calcular Todas las Liquidaciones

La lista de empleadas para liquidación aparecerá aquí.

 **Rendix**

Dashboard

Ventas

Gastos

Liquidaciones

Asistencias

Reportes

Importar

Configuración

Cerrar sesión

N

2 Issues

X

Dashboard > Asistencias

Exportar CSV

Gestión de Asistencias

Aquí se gestionará la asistencia de las vendedoras para el cálculo de presentismo.

👤

 Marcar Asistencia

El calendario o tabla de asistencias aparecerá aquí.

 **Rendix**

Dashboard

Ventas

Gastos

Liquidaciones

Asistencias

Reportes

Importar

Configuración

Cerrar sesión

N

2 Issues

×

Dashboard > Reportes

Exportar CSV

Generación de Reportes

Aquí se podrán generar y exportar reportes básicos en formato CSV o PDF.

Generar Reporte de Ventas

Las opciones para generar reportes aparecerán aquí.

 **Rendix**

Dashboard

Ventas

Gastos

Liquidaciones

Asistencias

Reportes

Importar

Configuración

Cerrar sesión

N

2 Issues

×

Dashboard > Importar

Importar Datos

Sube un archivo CSV para importar ventas o gastos masivamente.

Archivo CSV

Elegir archivo

No se ha seleccio...do ningún archivo

Subir y Procesar